

Λειτουργικά Συστήματα

Διδάσκων: Γεώργιος Ξυλωμένος

Εαρινό Εξάμηνο 2023 - 2024

Προγραμματιστική Εργασία

Συμμετέχοντες:

Παπαγεωργίου Παναγιώτης
p3220281

Καλαμά Δέσποινα
p3220060

Η εκτέλεση του προγράμματος δοκιμάστηκε στα περιβάλλοντα:

Lubuntu CSLab3Alt : x86_64

rasberry pi 5 : aarch64

Macbook M2 : aarch64

[p3220281-p3220060-pizza.h](#)

Στο header αρχείο αυτό οριστικοποιούμε τις σταθερές που μας έχουν ανατεθεί.

[test-res.sh](#)

Το shell script αρχείο αυτό εκτελεί τη μεταγλώττιση του C προγράμματος (τερματίζεται αν κάτι πάει λάθος στην μεταγλώττιση). Ο έλεγχος για την επιτυχή μεταγλώττιση αποτελείται από την ύπαρξη ή όχι του αρχείου “p3220281-p3220060-pizza” στο current working directory μας. Στη συνέχεια εκτελεί το εκτελέσιμο πρόγραμμα με παραμέτρους N_{cust} και Seed να λαμβάνουν τις τιμές τους από τις μεταβλητές CUSTOMER_COUNT=100 και INITIAL_SEED=10 που ορίζονται μέσα στο αρχείο. Καθόλη την διάρκεια εκτέλεσης ο χρήστης ενημερώνεται για την πρόοδο, με τα strings που εκτυπώνονται από τις εντολές echo στο standard output.

(Πριν το τρέξιμο του shell script θα πρέπει να δώσουμε τα κατάλληλα privileges για να το κάνουμε executable με το `chmod +x test-res.sh`)

```
(kali@kali-raspberry-pi5) - [~/Leitourgika]
$ ./test-res.sh
[+] Μεταγλώττιση προγράμματος: p3220281-p3220060-pizza.c...
[✓] Η μεταγλώττιση ολοκληρώθηκε.
Εκτέλεση προγράμματος με παραμέτρους: 100 πελάτες και αρχικό σπόρο 10.
-----
Διαθέσιμοι πόροι: Μάγειρες: 2, Φούρνοι: 10, Διανομείς: 10 | Ζωντανές παραγγελίες: 0
```

p3220281-p3220060-pizza.c

Στο αρχείο C βρίσκεται ο κώδικας για την εκτέλεση της προσομοίωσης της πιτσαρίας.

Η αρχιτεκτονική δομή του κώδικα είναι:

- Αρχικά κάνουμε include τα header αρχεία που χρειαζόμαστε. Εννοείται όλα τα βασικά headers για τις βασικές λειτουργίες, έπειτα το sys/ioctl.h καθώς το χρειαζόμαστε για το πρόσθετο χαρακτηριστικό που έχουμε υλοποιήσει. Τελικά το p3220281-p3220060-pizza.h για την επαφή στις σταθερές μας.
- Ορίζουμε το OrderData struct που χρησιμοποιείται για την αποθήκευση τιμών, χρήσιμες για κάθε παραγγελία. Αυτές είναι το id της, το seed της και το start_time στο οποίο αποθηκεύουμε άμεσα την χρονική στιγμή (αρχική εμφάνιση της παραγγελίας) για χρήση σε υπολογισμό χρονικών περιόδων διαδικασιών.
- Ορίζουμε τα mutex και conditions που χρειαζόμαστε για την ομαλή λειτουργία της πιτσαρίας για όλους τους παράγοντες: τηλεφωνητές, μάγειρες/παρασκευαστές κλπ.
- Ορίζουμε τις μεταβλητές στατιστικών που θα εμφανιστούν στο τέλος.
- Ορίζουμε την int get_terminal_height η οποία επιστρέφει το ύψος του τερματικού καθώς το χρειαζόμαστε για να εκτυπώνουμε στην τελευταία γραμμή χρήσιμες πληροφορίες για το monitoring της απασχόλησης του προσωπικού της πιτσαρίας μας. (Πήραμε υλικό από [αυτό το stackoverflow discussion](#))
- Ορίζουμε την `print_info` η οποία με την προηγούμενη συνάρτηση εκτυπώνει ένα monitoring string με χρήσιμες πληροφορίες (τους διαθέσιμους μάγειρες, τους ελεύθερους φούρνους, τους ελεύθερους διανομείς και τις παραγγελίες που βρίσκονται σε εξέλιξη. Θεωρούμε μια παραγγελία ζωντανή μέχρι να γυρίσει ο διανομέας στο μαγαζί καθώς το thread είναι ακόμα active). Όμως επειδή χρησιμοποιούμε το \n escape character, θα εκτυπώνονται από πάνω του τα updates των orders έτσι ώστε να φαίνεται σαν να εμφανίζεται μονο στη τελευταία γραμμή του τερματικού (επειδή επίσης κάνουμε συχνά update στην monitoring γραμμή αυτή).

Διαθέσιμοι πόροι: Μάγειρες: 2, Φούρνοι: 10, Διανομείς: 10 | Ζωντανές παραγγελίες: 0

```
[kali@kali-raspberrypi5] ~/Leitourgika
$ ./test-res.sh
[*] Μεταγλώττιση προγράμματος: p3220281-p3220060-pizza.c...
[*] Η μεταγλώττιση ολοκληρώθηκε.
Εκτέλεση προγράμματος με παραμέτρους: 100 πελάτες και αρχικό σπόρο 10.

[+] Η παραγγελία με αριθμό 1 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 2 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 3 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 4 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 5 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 6 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 1 ετοιμάστηκε σε 17 λεπτά.
[+] Η παραγγελία με αριθμό 7 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 2 ετοιμάστηκε σε 18 λεπτά.
[+] Η παραγγελία με αριθμό 8 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 9 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 3 ετοιμάστηκε σε 18 λεπτά.
[+] Η παραγγελία με αριθμό 1 παραδόθηκε σε 24 λεπτά.
[+] Η παραγγελία με αριθμό 10 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 4 ετοιμάστηκε σε 19 λεπτά.
[+] Η παραγγελία με αριθμό 2 παραδόθηκε σε 28 λεπτά.
[+] Η παραγγελία με αριθμό 11 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 5 ετοιμάστηκε σε 20 λεπτά.
[+] Η παραγγελία με αριθμό 3 παραδόθηκε σε 28 λεπτά.
[+] Η παραγγελία με αριθμό 12 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 4 παραδόθηκε σε 28 λεπτά.
[+] Η παραγγελία με αριθμό 13 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 6 ετοιμάστηκε σε 25 λεπτά.
[+] Η παραγγελία με αριθμό 14 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 7 ετοιμάστηκε σε 23 λεπτά.
[+] Η παραγγελία με αριθμό 5 παραδόθηκε σε 31 λεπτά.
[+] Η παραγγελία με αριθμό 15 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 7 παραδόθηκε σε 29 λεπτά.
[+] Η παραγγελία με αριθμό 8 ετοιμάστηκε σε 36 λεπτά.
[+] Η παραγγελία με αριθμό 6 παραδόθηκε σε 35 λεπτά.
[+] Η παραγγελία με αριθμό 10 ετοιμάστηκε σε 26 λεπτά.
[+] Η παραγγελία με αριθμό 12 ετοιμάστηκε σε 19 λεπτά.
[+] Η παραγγελία με αριθμό 16 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 9 ετοιμάστηκε σε 30 λεπτά.
[+] Η παραγγελία με αριθμό 18 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 17 καταχωρήθηκε.
[+] Η παραγγελία με αριθμό 19 καταχωρήθηκε.

Διαθέσιμοι πόροι: Μάγειρες: 0, Φούρνοι: 0, Διανομείς: 5 | Ζωντανές παραγγελίες: 12
```

Για παράδειγμα εδώ ενημερώνει η γραμμή ότι υπάρχουν 12 ζωντανές παραγγελίες όσα είναι και τα threads του προγράμματος όπως βλέπουμε στο htop utility (εκτός του 1ου main thread).

- **Σημείωση: το πρόγραμμα απαιτεί πλάτος terminal τουλάχιστον 85 χαρακτήρων.** -

- Στη συνέχεια είναι η [handle_order](#), παίρνει void pointer arguments καθώς χρησιμοποιείται στην δημιουργία των pthreads. Το void pointer arg θα γίνει casted σε OrderData* από το οποίο θα λάβουμε τα στοιχεία της παραγγελίας από τον χώρο που κάναμε allocate για αυτή την παραγγελία στην main. Έπειτα για την όλη διαδικασία της παραγγελίας τηρούνται αυτά τα βήματα:
 - Βήμα 1:
Περιμένουμε να βρούμε τηλεφωνητή για την παραγγελία κάνοντας πρώτα lock έπειτα wait, για να εμφανιστεί ένας διαθέσιμος τηλεφωνητής, και έπειτα unlock.
 - Βήμα 2:
Λαμβάνουμε τον αριθμό από πίτσες που θέλει η παραγγελία. Χρησιμοποιούμε το rand_r με το seed της παραγγελίας (που από παραγγελία σε παραγγελία διαφέρει) και τις μεταβλητές Norderhigh και Norderlow.
 - Βήμα 3:
Έλεγχος για αποτυχία χρέωσης και άρα ακύρωσης παραγγελίας. Χρήση της sleep για την χρέωση της πιστωτικής κάρτας του πελάτη.
 - Βήμα 4:
Αν γίνει αποτυχία πληρωμής (με βάση το Pfail) ενημερώνουμε τον τερματικό καθώς και το monitoring line (κάνοντας lock και unlock το print mutex για την ομαλή εκτύπωση). Χρησιμοποιούμε την συνάρτηση __sync_fetch_and_add για το Increment του Counter των αποτυχημένων παραγγελιών.
 - Βήμα 5:
Η παραγγελία συνεχίζεται και με βάση τα Cm, Cp, Cs υπολογίζουμε τα έσοδα από τις πίτσες της παραγγελίας. Ταυτόχρονα ενημερώνουμε τις στατιστικές μεταβλητές μας. Εκτυπώνουμε ότι η παραγγελία καταχωρήθηκε καθώς και την monitoring γραμμή. Και τελικά ενημερώνουμε ότι ελευθερώθηκε ο τηλεφωνητής.
 - Βήμα 6:
Περιμένουμε για μάγαιρα/παρασκευαστή (Χρησιμοποιούμε βρόχο while για να ελέγχει την ύπαρξη μάγαιρα και κάνουμε pthread_cond_wait μέχρι τότε). Όταν βρεθεί τον μειώνουμε από τους διαθέσιμους μαγειρες και ενημερώνουμε και την monitoring γραμμή).
 - Βήμα 7:
Ο μάγαιρας/παρασκευαστής ετοιμάζει τις πίτσες (περιμένουμε με την sleep για Tprep, για κάθε πίτσα).
 - Βήμα 8:
Περιμένουμε όσους φούρνους όσες έχουμε πίτσες κάνοντας lock και unlock την oven_mutex και περιμένοντας την oven_cond. Έπειτα ανανεώνουμε την monitoring γραμμή.
 - Βήμα 9:
Περιμένουμε με την sleep Tbake χρόνο για το ψήσιμο των πιτσών. Πριν αυτού, ο μάγαιρας/παρασκευαστής παύει να ασχολείται με την παραγγελία αυτή και προχωρά στην επόμενη. Κάνουμε ενημέρωση της monitoring γραμμής.
 - Βήμα 10:

Περιμένουμε για διαθέσιμο διανομέα. Με το που έχουμε διαθέσιμο διανομέα απελευθερώνουμε όσους φούρνους χρησιμοποιούσαμε. Κάνουμε ενημέρωση της monitoring γραμμής. Και ενημερώνουμε ότι ένας διανομέας τώρα θα είναι απασχολημένος με την τρέχουσα παραγγελία.

- Βήμα 11:

Ο διανομέας πακετάρει τις πίτσες της παραγγελίας (χρόνος: $T_{pack} * num_pizzas$). Ενημερώνουμε ότι η παραγγελία βρίσκεται σε κατάσταση “έτοιμη” καθώς υπολογίζουμε και εκτυπώνουμε την ώρα από την εμφάνιση της παραγγελίας μέχρι τώρα. Επίσης ενημερώνουμε την monitoring line.

- Βήμα 12:

Ο διανομέας μεταφέρει τις πίτσες από τη πιτσαρία σε χρόνο που υπολογίζουμε με την $rand_r$ το seed και τα όρια T_{dellow} και $T_{delhigh}$. Έχουμε sleep για την διαδρομή του διανομέα προς τον πελάτη. Υπολογίζουμε την ώρα από την εμφάνιση της παραγγελίας και εκτυπώνουμε το μήνυμα με την ολοκλήρωση της παραγγελίας.

- Βήμα 13:

Ο διανομέας επιστρέφει στη πιτσαρία σε χρόνο ίδιο όσο έκανε και για να πάει στον πελάτη. Κάνουμε lock ενημερώνουμε ότι είναι ελεύθερος και κάνουμε πάλι unlock. Υπολογίζουμε τους χρόνους εξυπηρέτησης και κρυώματος παραγγελιών. Τελικά κάνουμε lock το `stats_mutex` για να ενημερώσουμε τα global stats και έπειτα unlock, κάνουμε increment τα `total_successful_orders` με την `__sync_fetch_and_add` και ολοκληρώθηκε η παραγγελία.

Αποδεσμεύουμε τον χώρο μνήμης της παραγγελίας.

- Τελικά στην `main` πρώτα δεχόμαστε και μετατρέπουμε τις `*char` παραμέτρους `Ncust` και `Seed` από το terminal και κάνουμε initialise την `srand` με το `Seed`. Αποθηκεύουμε την χρονική στιγμή και την θεωρούμε αρχική. Έχουμε ένα for loop που κανουμε allocate με την `malloc` χώρο μεγέθους `sizeof(OrderData)` (αν αυτό είναι δυνατόν) και έπειτα γίνεται προσπάθεια δημιουργίας pthread έχοντας κάνει initialise τις μεταβλητές του `OrderData struct`. Τέλος κάνουμε join όλα τα threads σε ένα ξεχωριστό for loop και τελικά εμφανίζονται τα στατιστικά του προγράμματος.